Event Storming: Oficina Mecânica

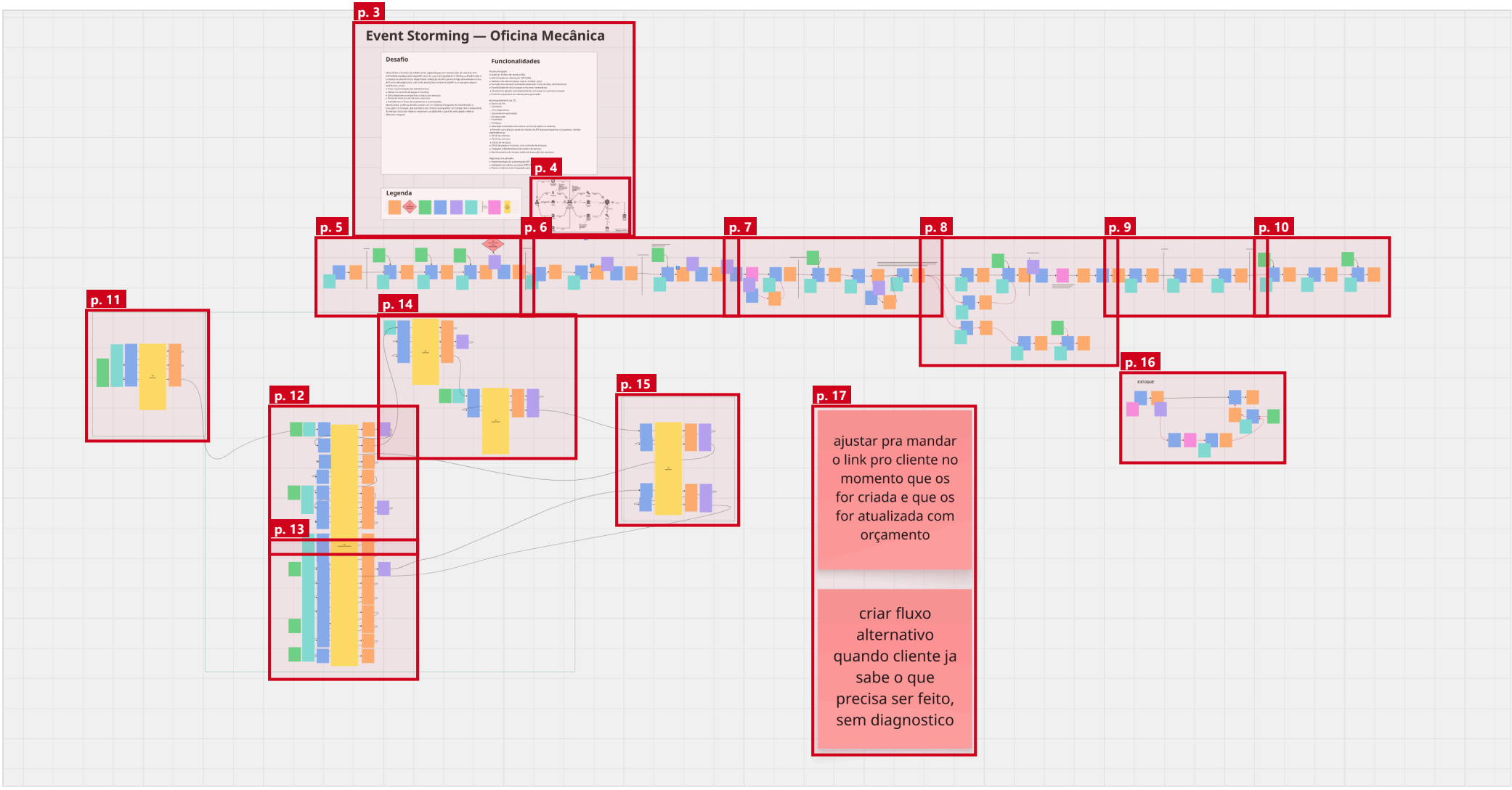
Tech Challenge SOAT, fase 1, grupo 31.

- **Parte A: Event Storming exploratório (Miro)**, páginas 2 a 21. Exportação do board "Oficina Mecânica System" (https://miro.com/app/board/uXjVHnerRBQ=/), feito pelo grupo no início do projeto: mapa do board (página 2), capturas em alta resolução de cada região (páginas 3 a 17) e transcrição das notas (páginas 18 a 21).
- **Parte B: Event Storming do sistema implementado**, páginas 22 a 26. As três linhas do tempo do enunciado (criação, acompanhamento, peças e insumos) com os comandos, eventos, políticas, atores e agregados reais do código.

O board exploratório tem termos e passos que o código tratou de outro jeito (mecânico operando o sistema, WhatsApp, PDF, ordem de compra, relatório). A relação item a item está em `docs/ddd/linguagem-ubiqua.md`, seção "Relação com o código".

A. Event Storming exploratório (Miro): mapa do board

Visão inteira do board. Cada retângulo vermelho indica a página em que aquela região aparece ampliada.



A. Desafio, funcionalidades e legenda

Captura do board no Miro. Legenda do board: laranja = evento (ev), azul = comando (cmd), lilás = política (pol), verde = modelo de leitura (ml), turquesa = ator (at), rosa = sistema externo (se), amarelo = agregado (ag), losango vermelho = ponto de atenção (pa).

Event Storming — Oficina Mecânica

Desafio

Uma oficina mecânica de médio porte, especializada em manutenção de veículos, tem enfrentado desafios para expandir seus serviços com qualidade e eficiência. Atualmente, o processo de atendimento, diagnóstico, execução de serviços e entrega dos veículos é feito de forma desorganizada, utilizando anotações manuais e planilhas, o que gera alguns problemas, como:

- Erros na priorização dos atendimentos;
- Falhas no controle de peças e insumos;
- Dificuldade em acompanhar o status dos serviços;
- Perda de histórico de clientes e veículos;
- Ineficiência no fluxo de orçamentos e autorizações.

Diante disso, a oficina decidiu investir em um Sistema Integrado de Atendimento e Execução de Serviços, que permitirá aos clientes acompanhar em tempo real o andamento do serviço, autorizar reparos adicionais via aplicativo e garantir uma gestão interna eficiente e segura.

Funcionalidades

Fluxos principais
Criação da Ordem de Serviço (OS):

- Identificação do cliente por CPF/CNPJ;
- Cadastro de veículo (placa, marca, modelo, ano);
- Inclusão dos serviços solicitados (exemplo: troca de óleo, alinhamento);
- Possibilidade de incluir peças e insumos necessários;
- Orçamento gerado automaticamente com base nos serviços e peças;
- Envio do orçamento ao cliente para aprovação.

Acompanhamento da OS:

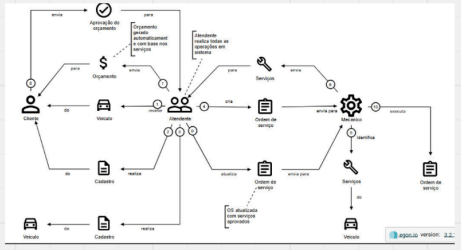
- Status da OS:
  - Recebida
; - Em diagnóstico;
  - Aguardando aprovação;
- Em execução;
- Finalizada;
- Entregue.
- Alteração automática dos status conforme ações no sistema;
- Permitir consulta por parte do cliente via API para acompanhar o progresso. Gestão administrativa:
- CRUD de clientes;
- CRUD de veículos;
- CRUD de serviços;
- CRUD de peças e insumos, com controle de estoque;
- Listagem e detalhamento de ordens de serviço;
- Monitoramento do tempo médio de execução dos serviços.

Segurança e qualidade:

- Implementação de autenticação JWT para APIs administrativas;
- Validação dos dados sensíveis (CPF/CNPJ, placa de veículo);
- Testes unitários e de integração para os principais fluxos

Legenda

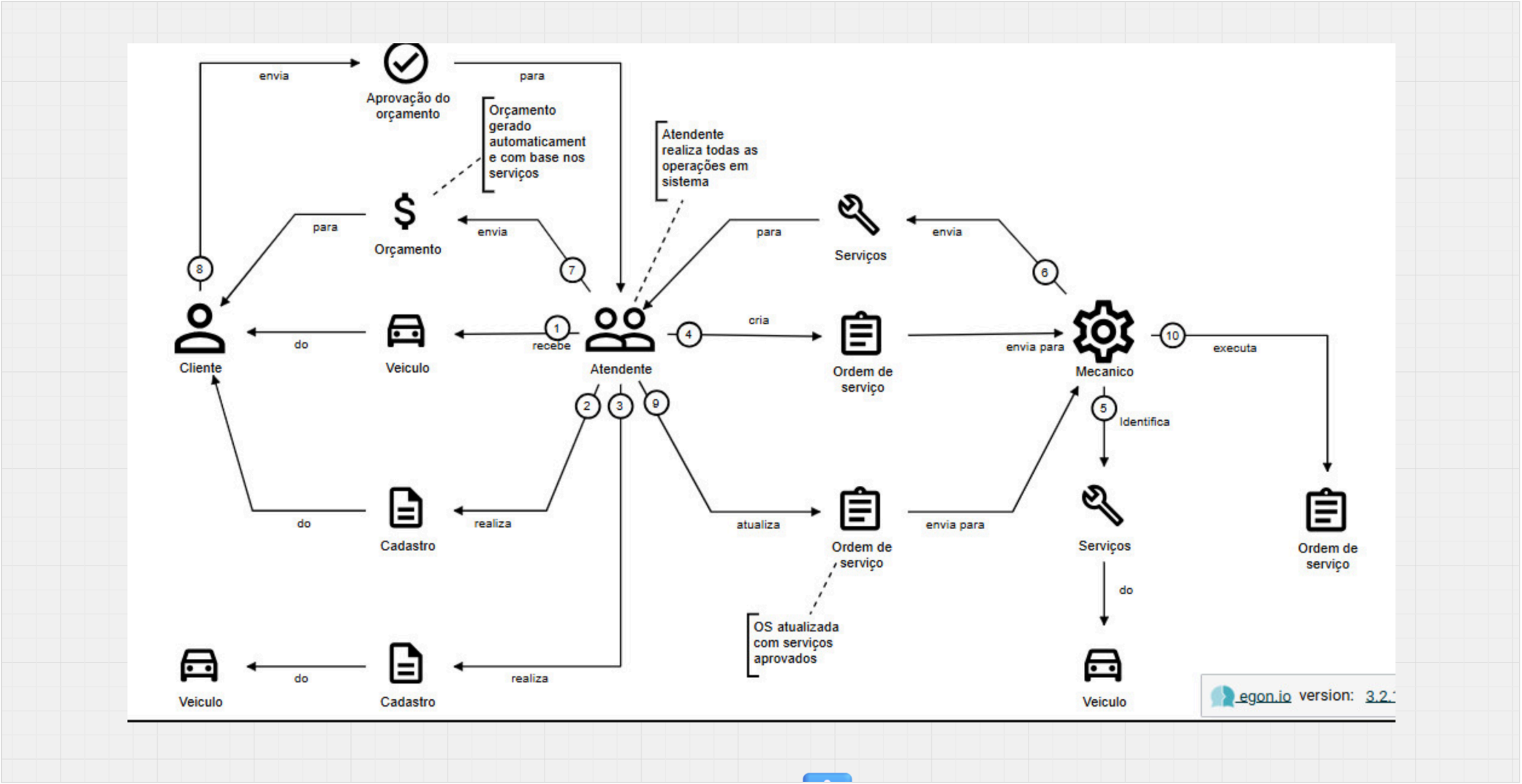






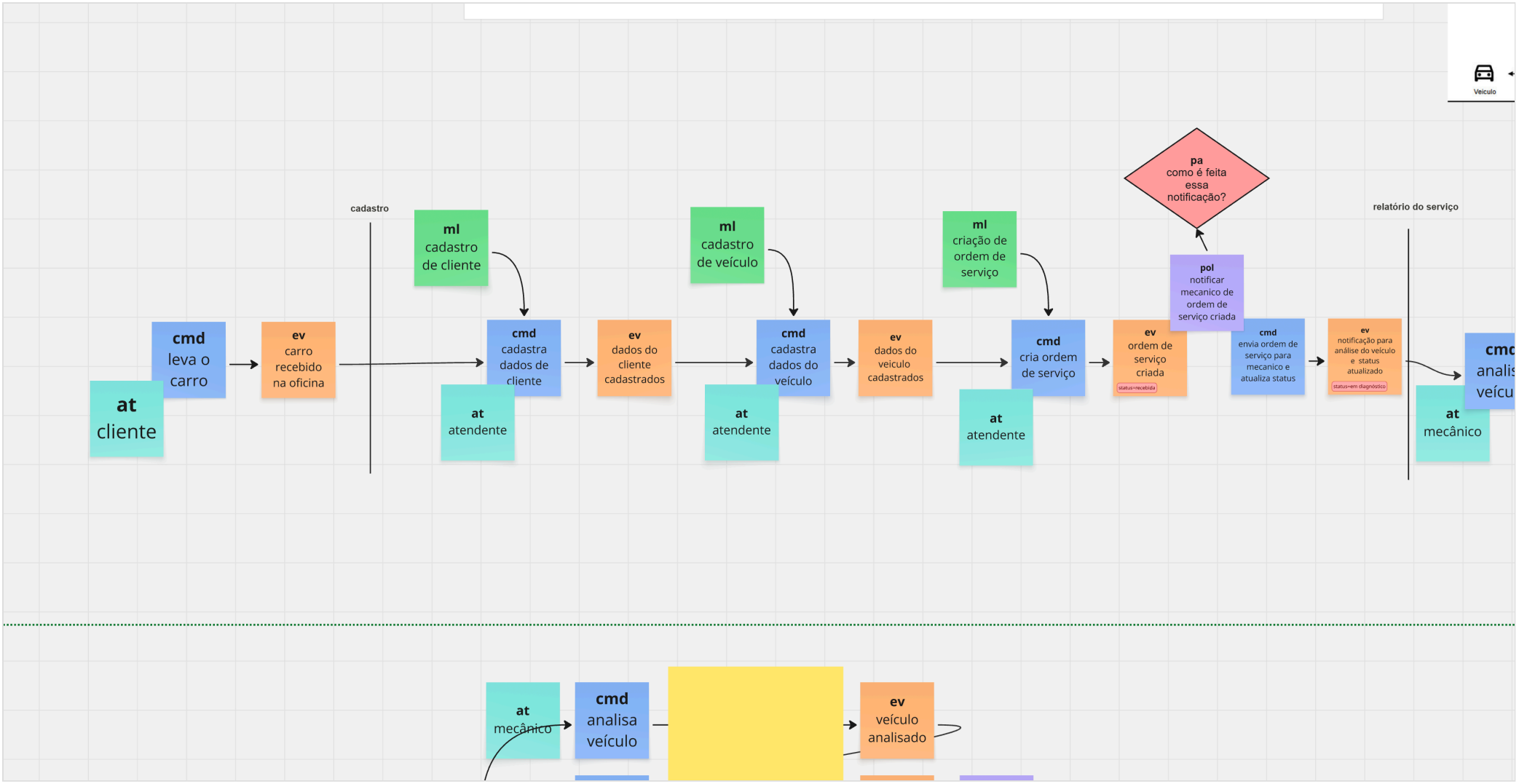
A. Mapa de atores e objetos (rascunho do grupo)

Captura do board no Miro. Legenda do board: laranja = evento (ev), azul = comando (cmd), lilás = política (pol), verde = modelo de leitura (ml), turquesa = ator (at), rosa = sistema externo (se), amarelo = agregado (ag), losango vermelho = ponto de atenção (pa).



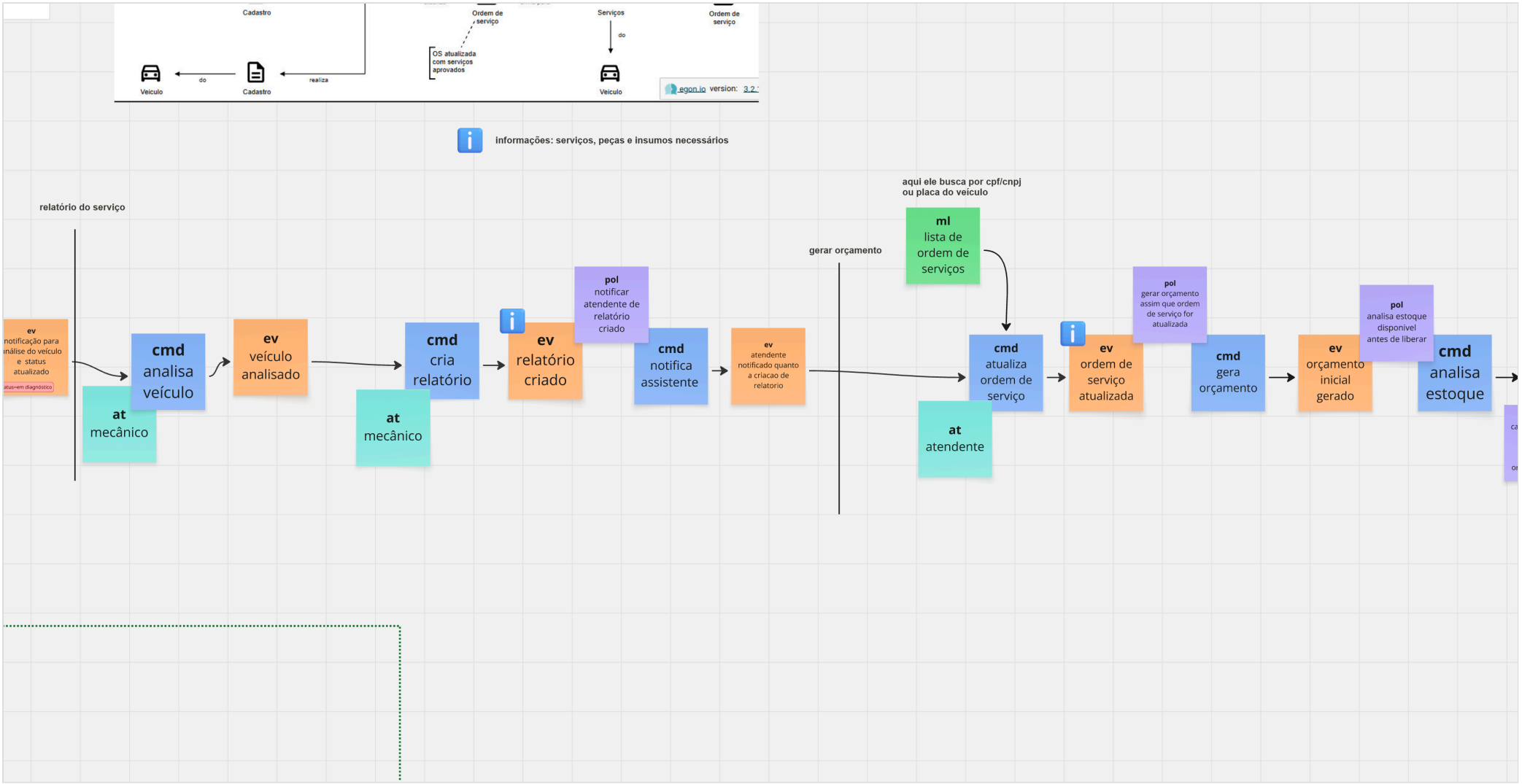
A. Linha do tempo 1/6: recepção, cadastro e criação da OS

Captura do board no Miro. Legenda do board: laranja = evento (ev), azul = comando (cmd), lilás = política (pol), verde = modelo de leitura (ml), turquesa = ator (at), rosa = sistema externo (se), amarelo = agregado (ag), losango vermelho = ponto de atenção (pa).



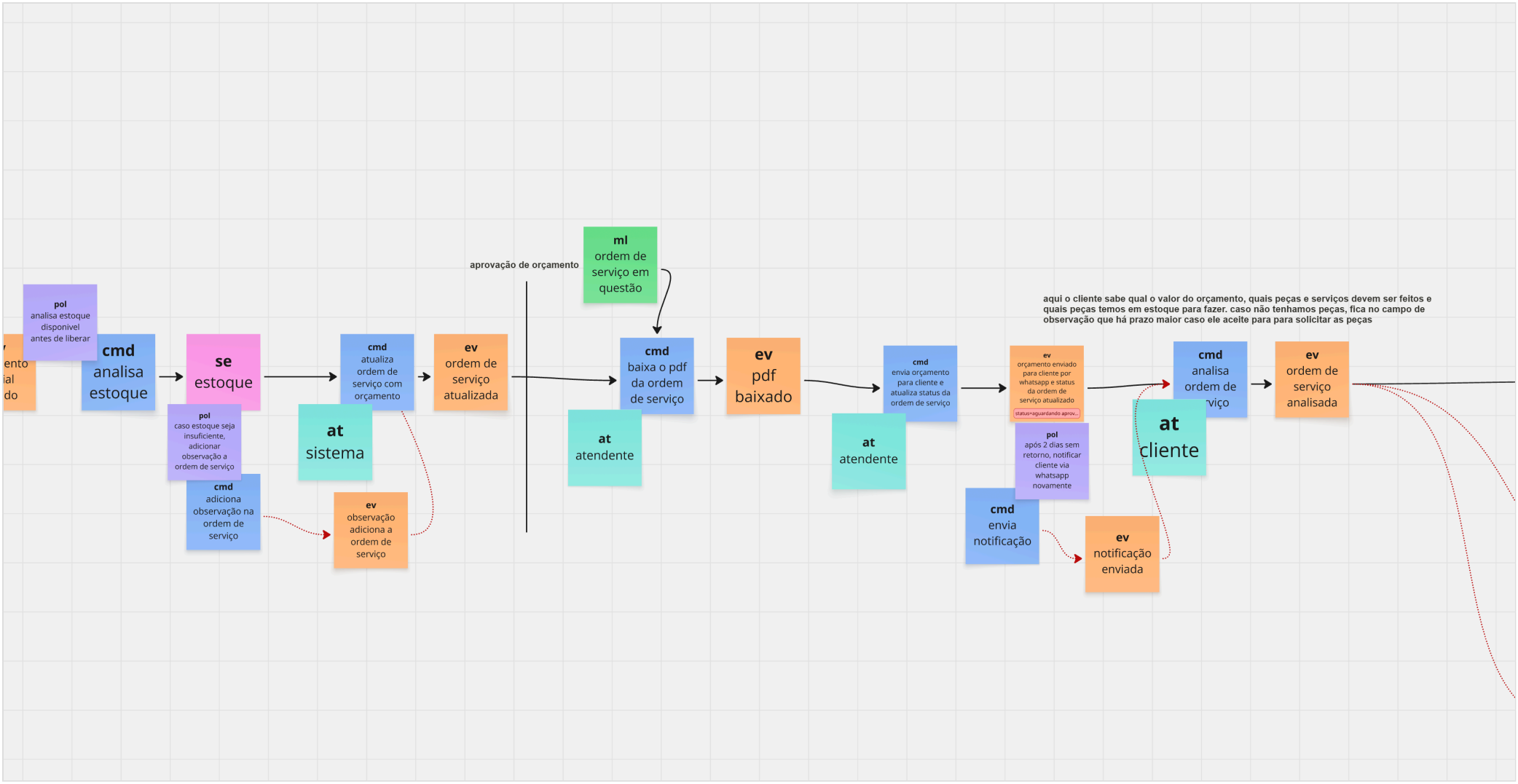
A. Linha do tempo 2/6: diagnóstico, relatório e geração do orçamento

Captura do board no Miro. Legenda do board: laranja = evento (ev), azul = comando (cmd), lilás = política (pol), verde = modelo de leitura (ml), turquesa = ator (at), rosa = sistema externo (se), amarelo = agregado (ag), losango vermelho = ponto de atenção (pa).



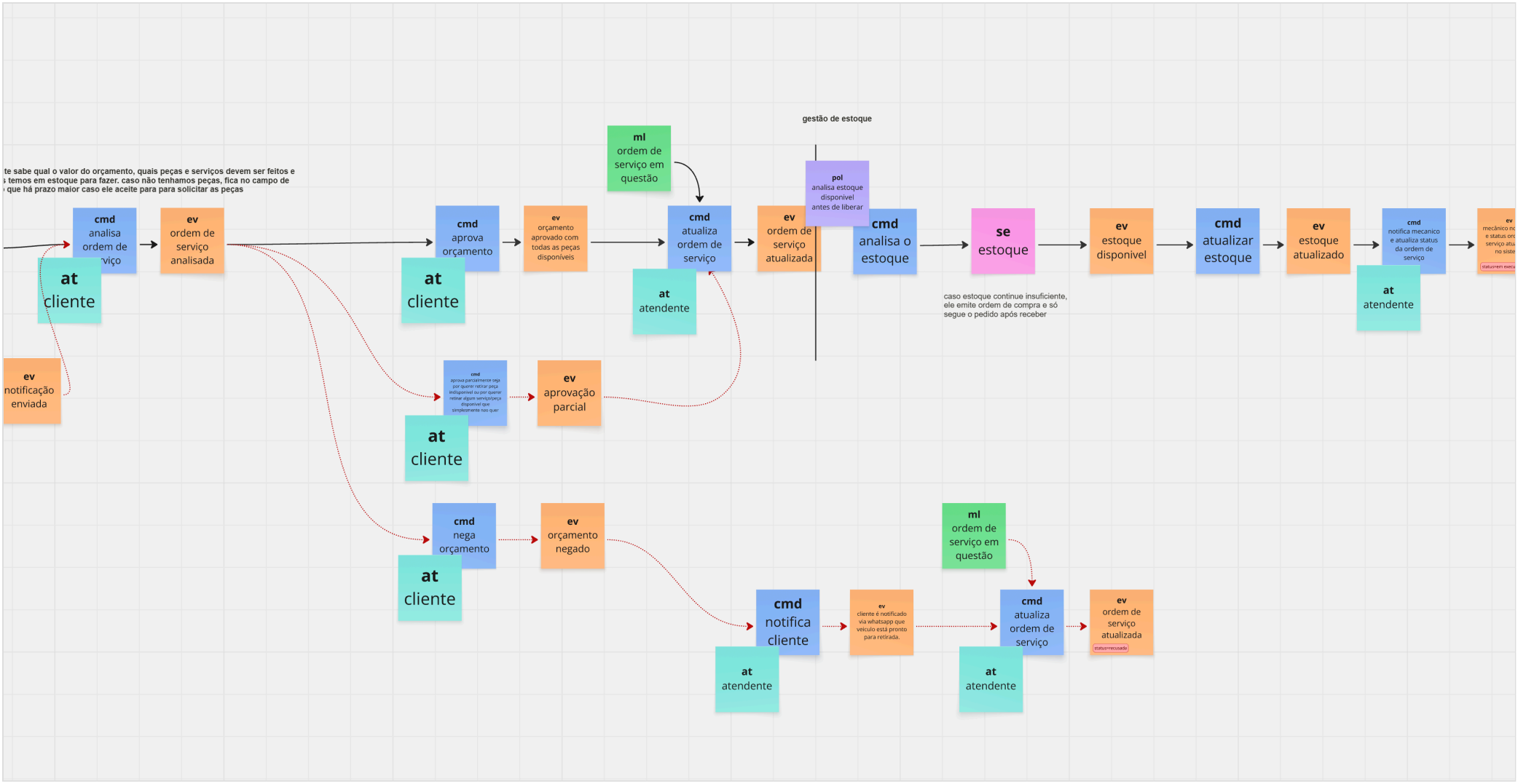
A. Linha do tempo 3/6: estoque, PDF e envio do orçamento ao cliente

Captura do board no Miro. Legenda do board: laranja = evento (ev), azul = comando (cmd), lilás = política (pol), verde = modelo de leitura (ml), turquesa = ator (at), rosa = sistema externo (se), amarelo = agregado (ag), losango vermelho = ponto de atenção (pa).



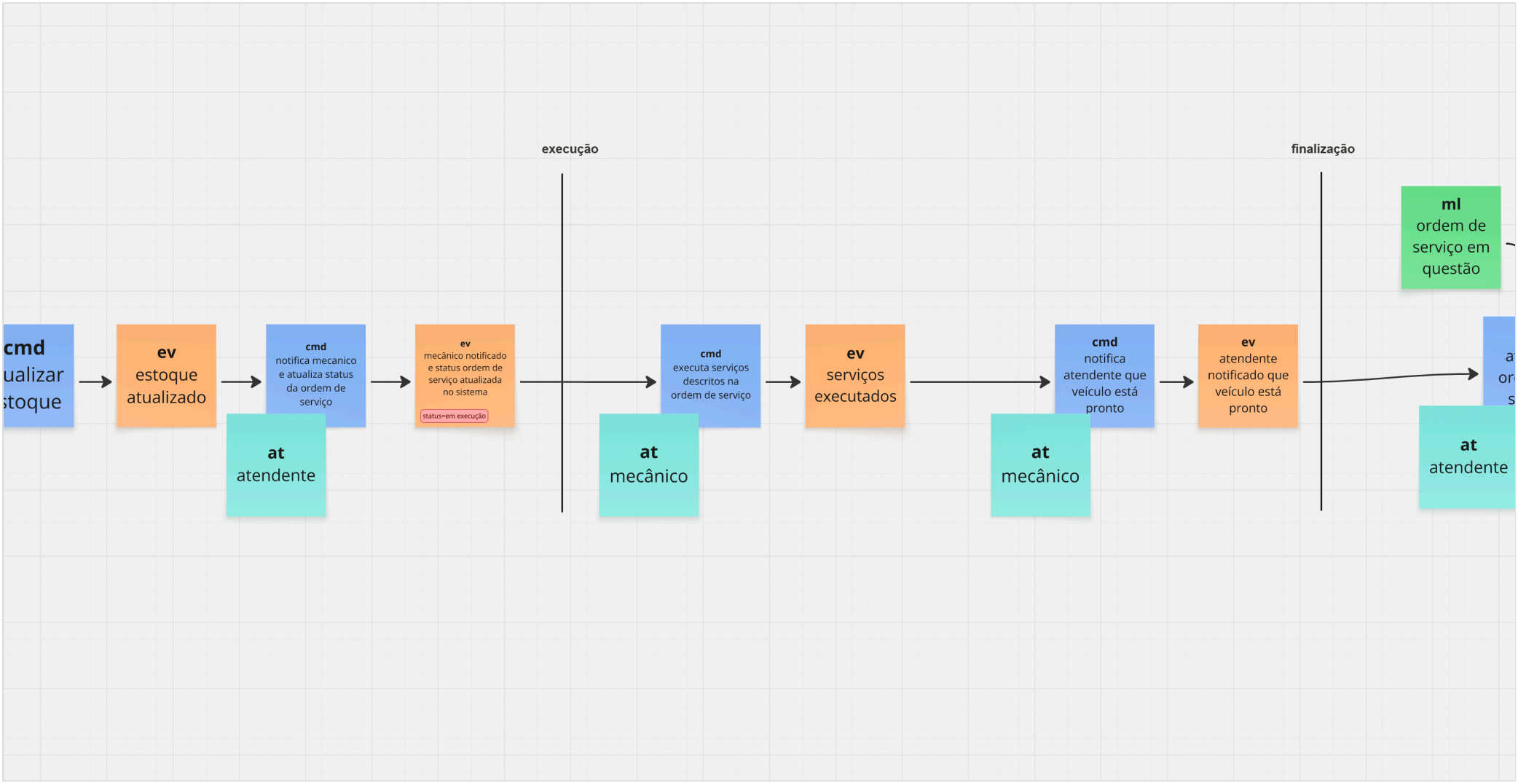
A. Linha do tempo 4/6: aprovação, aprovação parcial, reprovação e gestão de estoque

Captura do board no Miro. Legenda do board: laranja = evento (ev), azul = comando (cmd), lilás = política (pol), verde = modelo de leitura (ml), turquesa = ator (at), rosa = sistema externo (se), amarelo = agregado (ag), losango vermelho = ponto de atenção (pa).



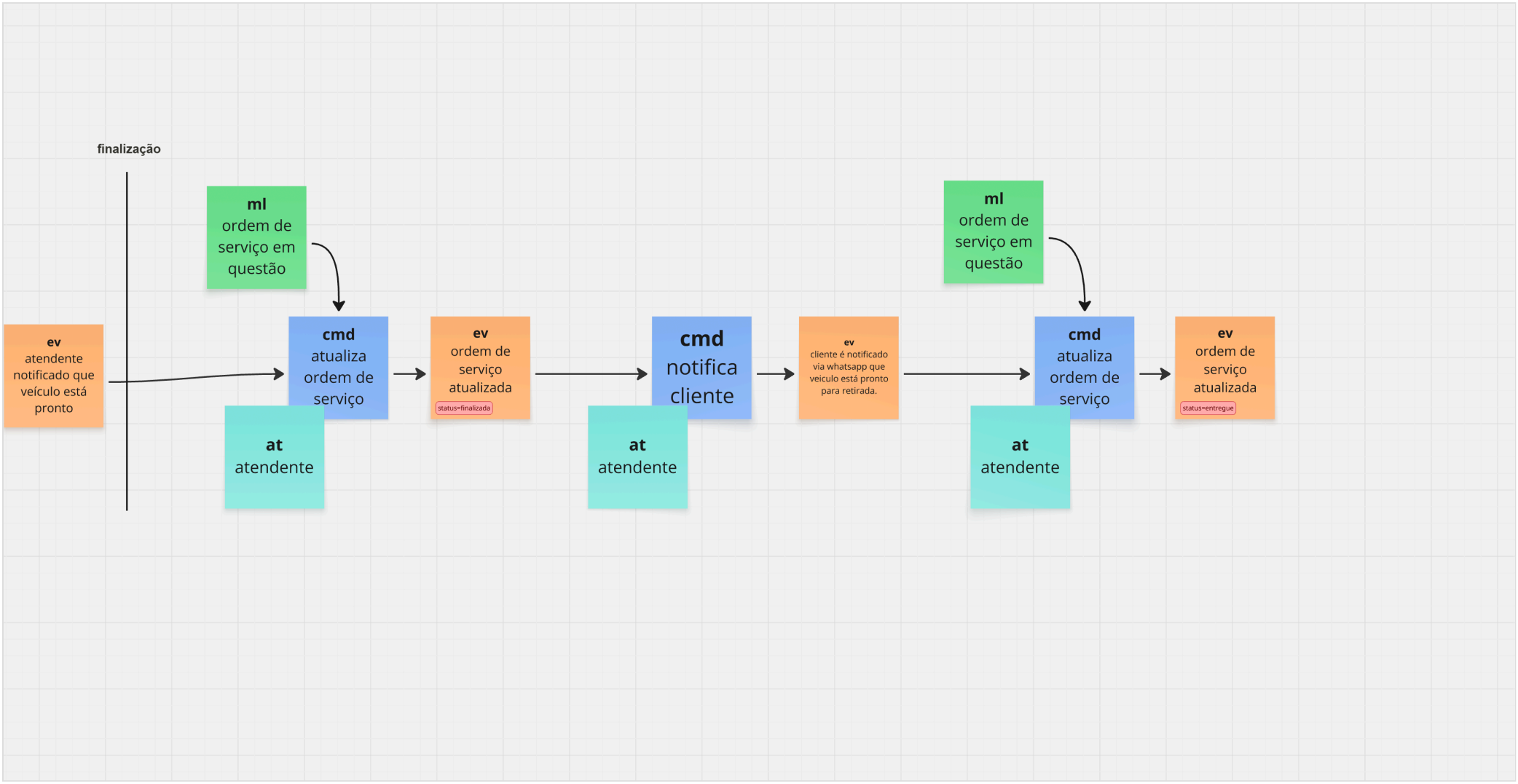
A. Linha do tempo 5/6: estoque atualizado, execução e aviso de veículo pronto

Captura do board no Miro. Legenda do board: laranja = evento (ev), azul = comando (cmd), lilás = política (pol), verde = modelo de leitura (ml), turquesa = ator (at), rosa = sistema externo (se), amarelo = agregado (ag), losango vermelho = ponto de atenção (pa).



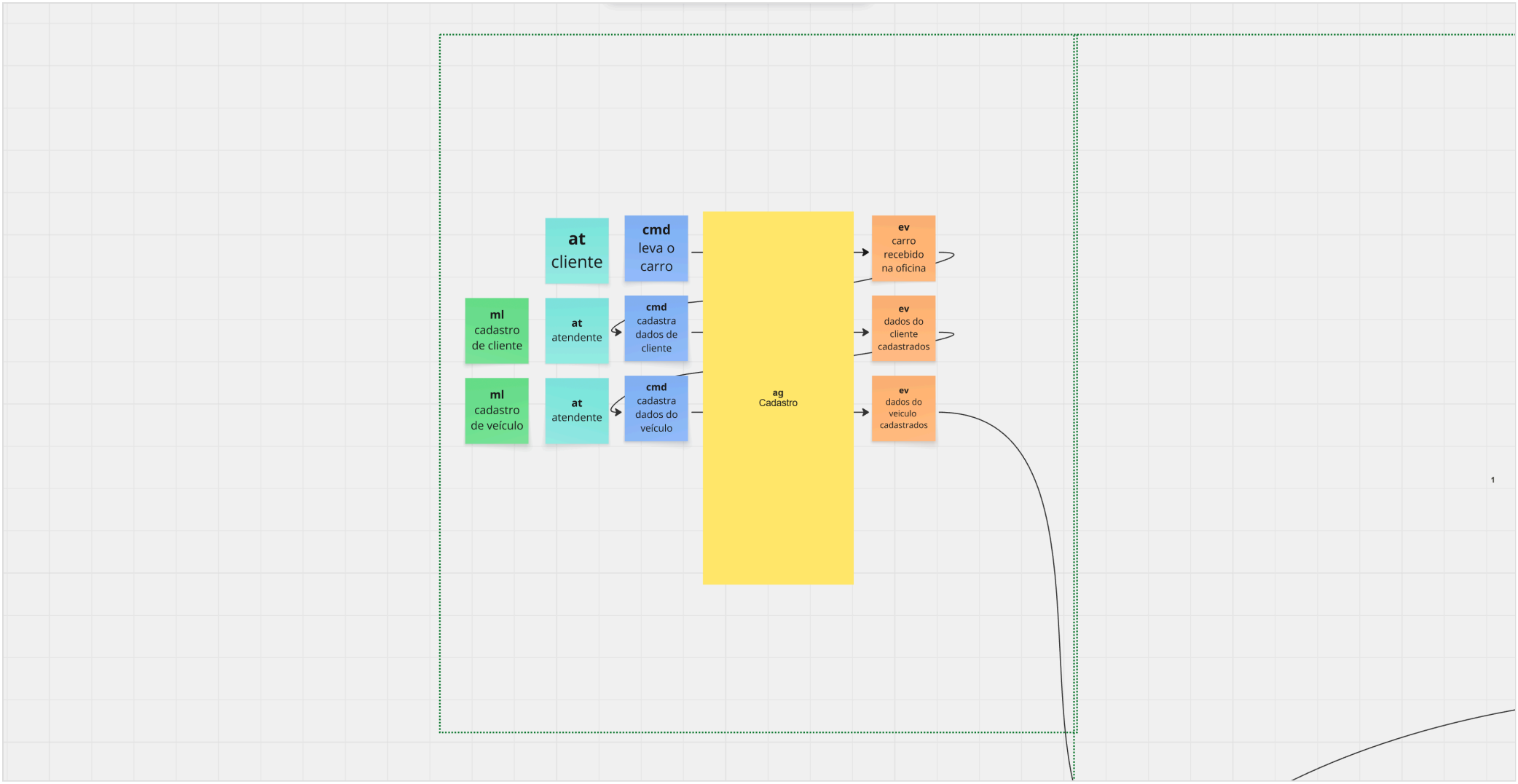
A. Linha do tempo 6/6: finalização e entrega

Captura do board no Miro. Legenda do board: laranja = evento (ev), azul = comando (cmd), lilás = política (pol), verde = modelo de leitura (ml), turquesa = ator (at), rosa = sistema externo (se), amarelo = agregado (ag), losango vermelho = ponto de atenção (pa).



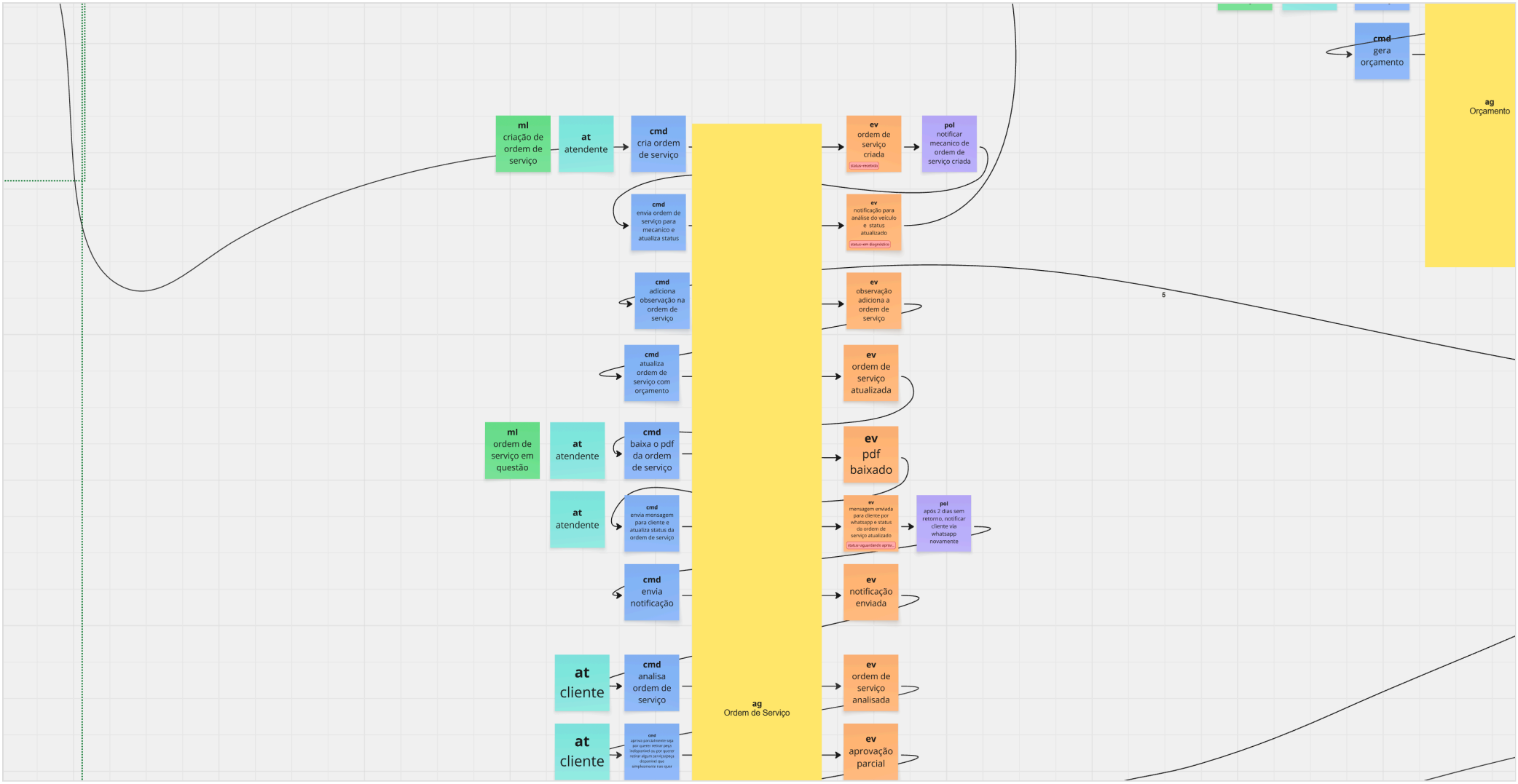
A. Agregado Cadastro

Captura do board no Miro. Legenda do board: laranja = evento (ev), azul = comando (cmd), lilás = política (pol), verde = modelo de leitura (ml), turquesa = ator (at), rosa = sistema externo (se), amarelo = agregado (ag), losango vermelho = ponto de atenção (pa).



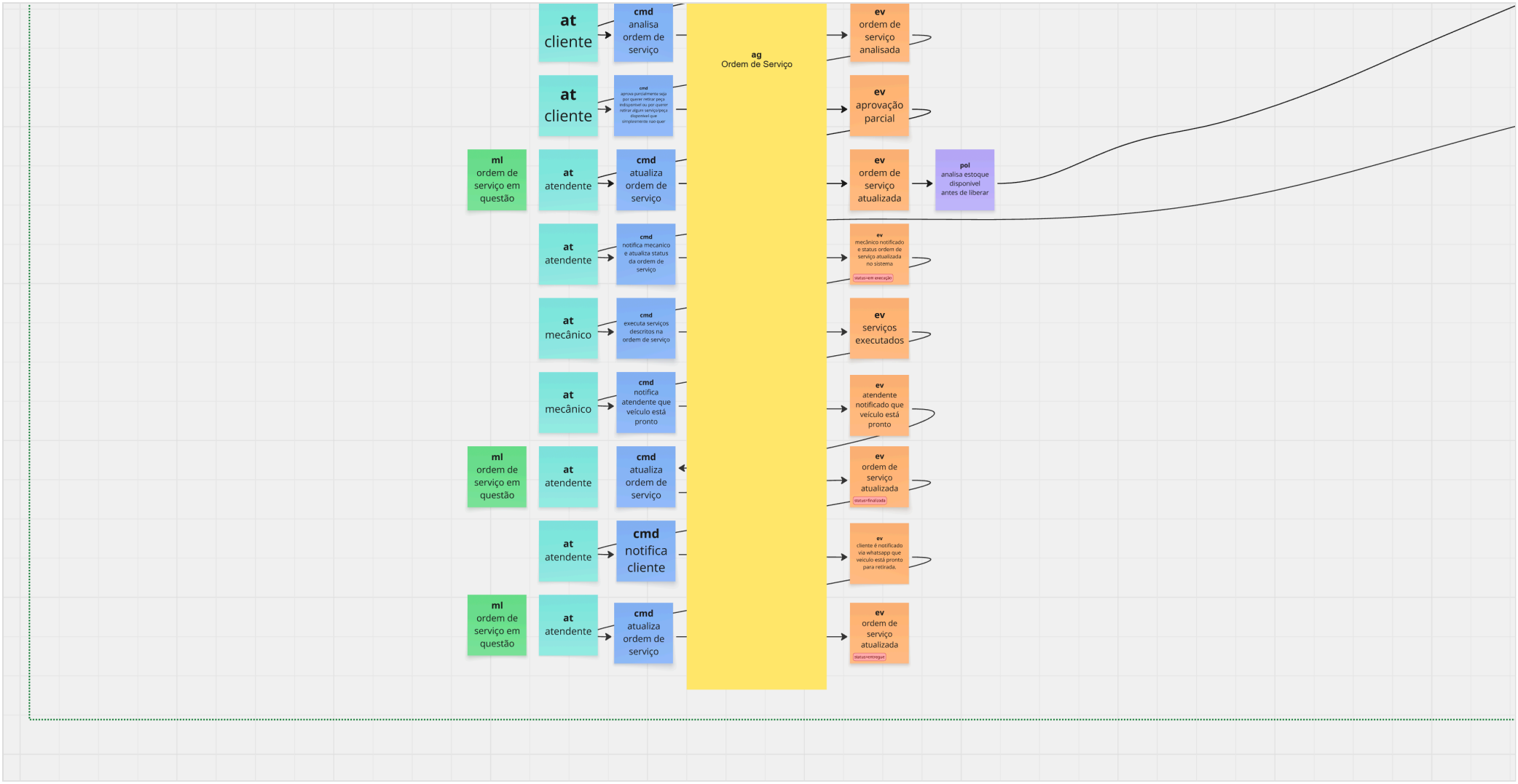
A. Agregado Ordem de Serviço (parte 1)

Captura do board no Miro. Legenda do board: laranja = evento (ev), azul = comando (cmd), lilás = política (pol), verde = modelo de leitura (ml), turquesa = ator (at), rosa = sistema externo (se), amarelo = agregado (ag), losango vermelho = ponto de atenção (pa).



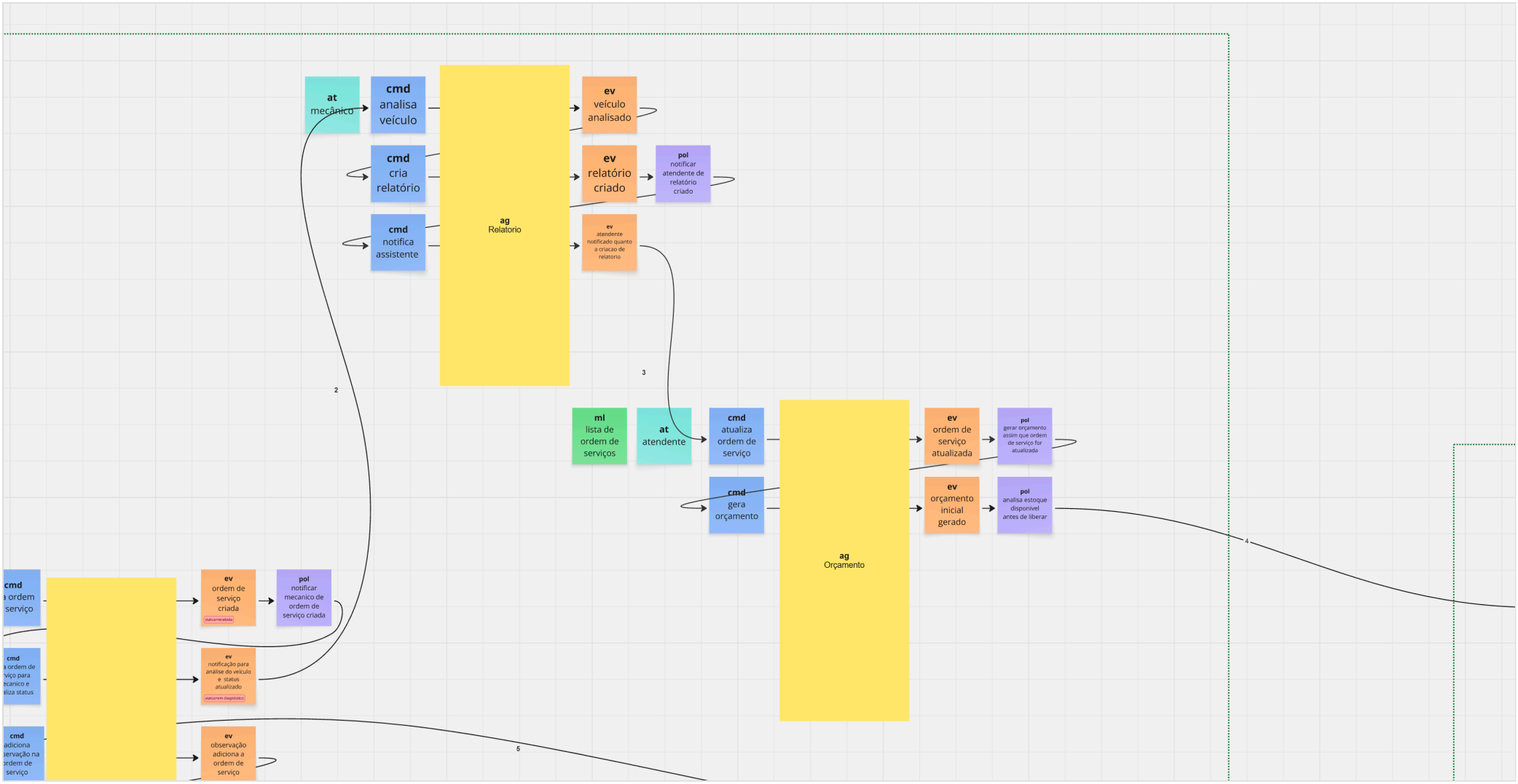
A. Agregado Ordem de Serviço (parte 2)

Captura do board no Miro. Legenda do board: laranja = evento (ev), azul = comando (cmd), lilás = política (pol), verde = modelo de leitura (ml), turquesa = ator (at), rosa = sistema externo (se), amarelo = agregado (ag), losango vermelho = ponto de atenção (pa).



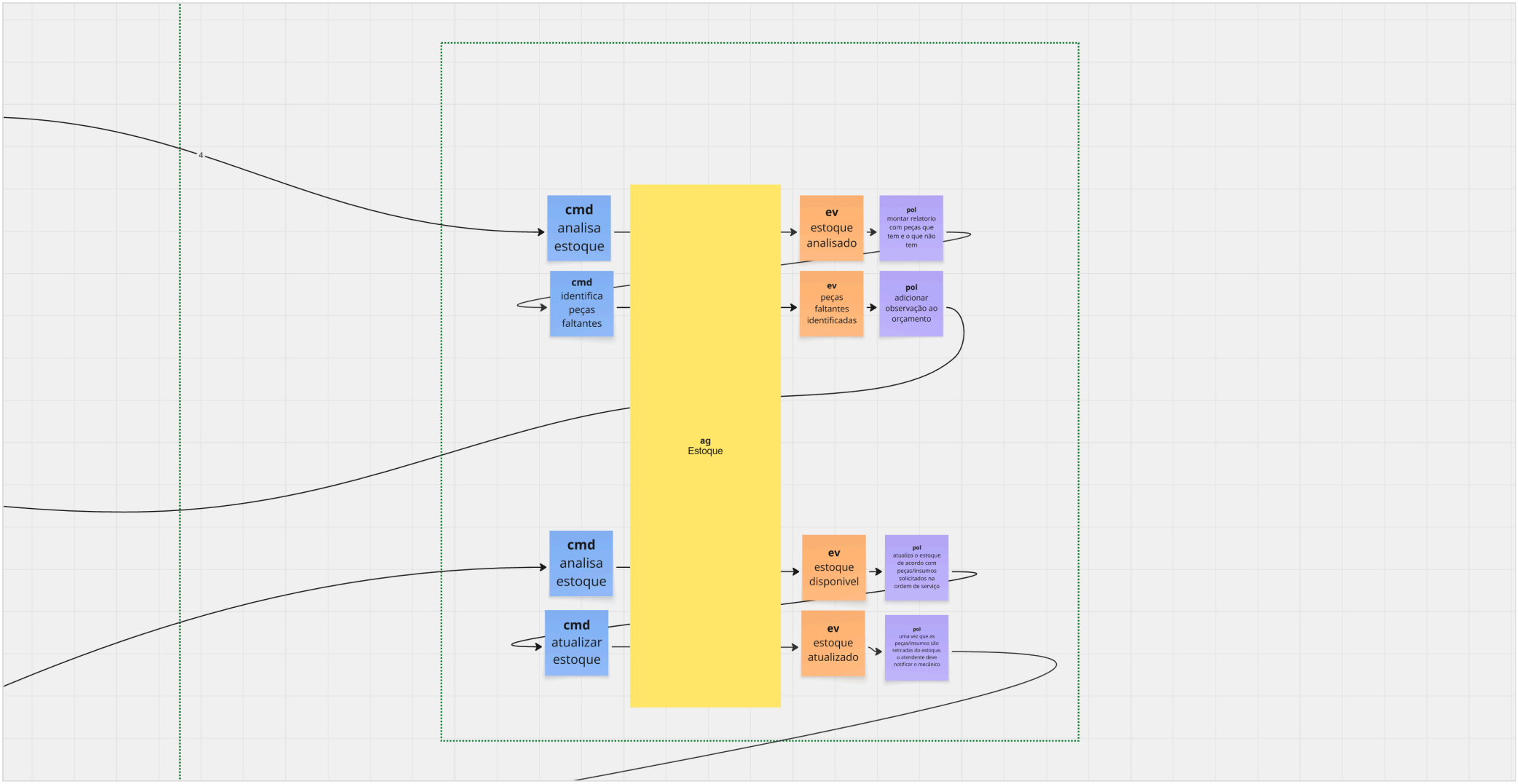
A. Agregados Relatório e Orçamento

Captura do board no Miro. Legenda do board: laranja = evento (ev), azul = comando (cmd), lilás = política (pol), verde = modelo de leitura (ml), turquesa = ator (at), rosa = sistema externo (se), amarelo = agregado (ag), losango vermelho = ponto de atenção (pa).



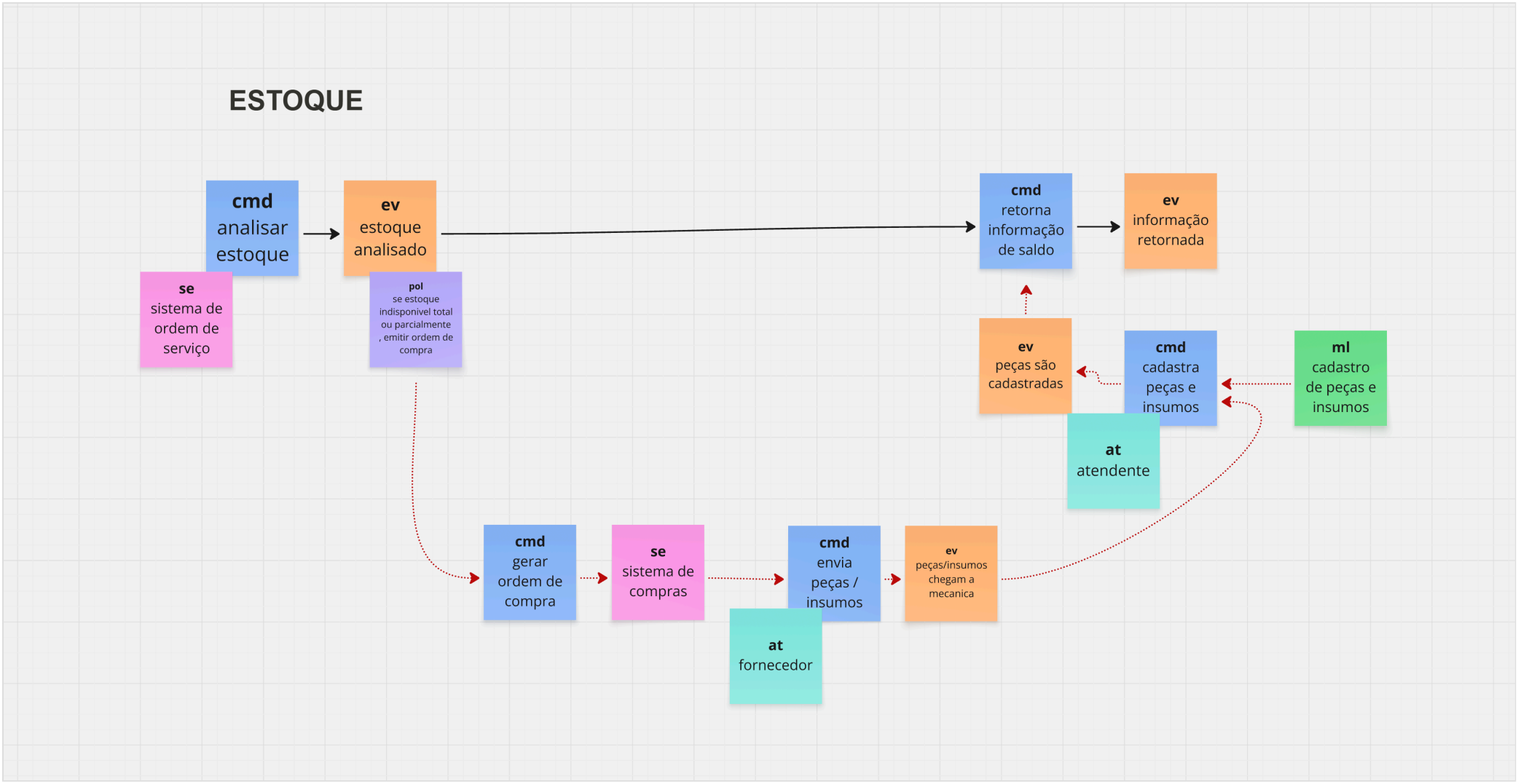
A. Agregado Estoque

Captura do board no Miro. Legenda do board: laranja = evento (ev), azul = comando (cmd), lilás = política (pol), verde = modelo de leitura (ml), turquesa = ator (at), rosa = sistema externo (se), amarelo = agregado (ag), losango vermelho = ponto de atenção (pa).



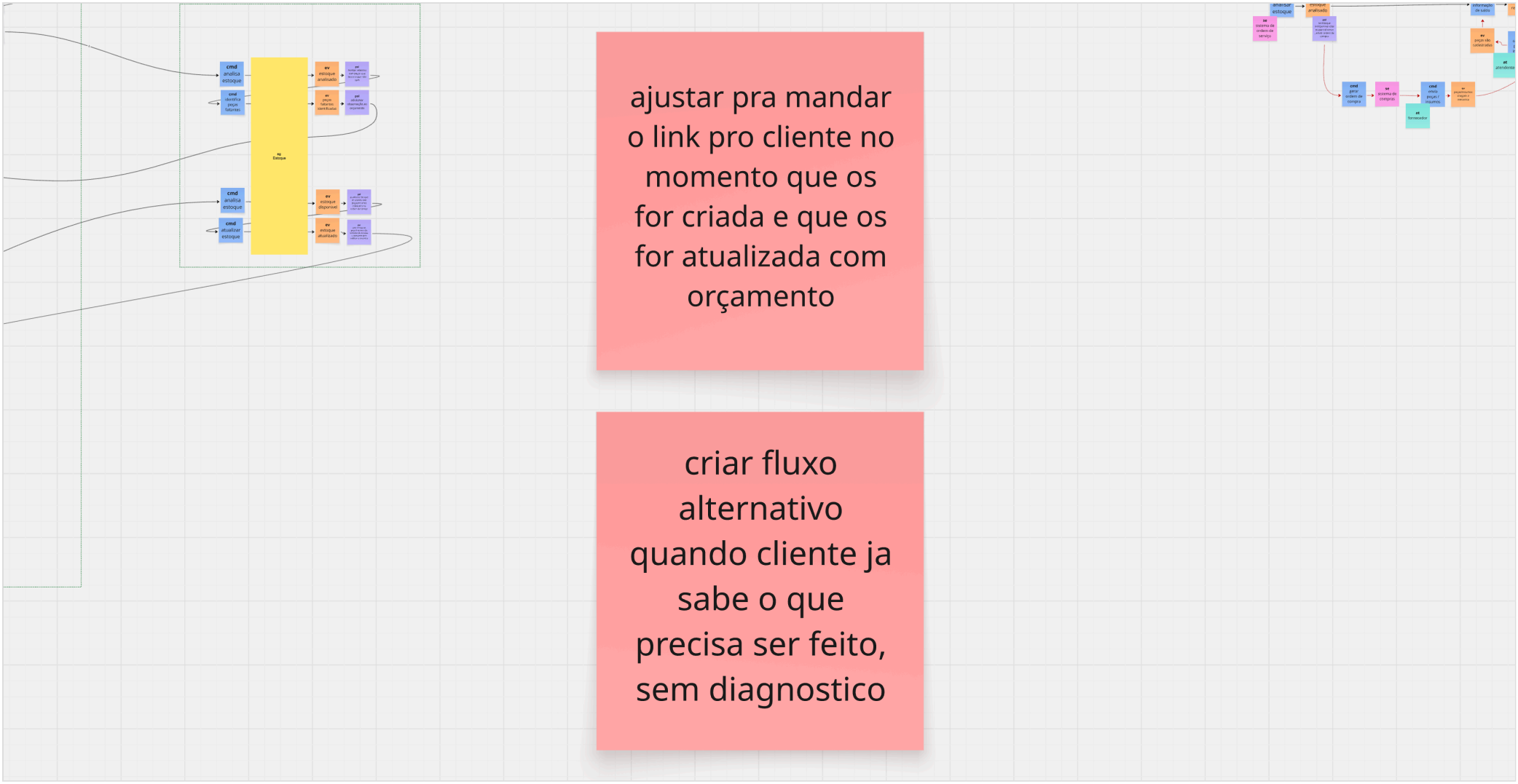
A. Fluxo de peças e insumos

Captura do board no Miro. Legenda do board: laranja = evento (ev), azul = comando (cmd), lilás = política (pol), verde = modelo de leitura (ml), turquesa = ator (at), rosa = sistema externo (se), amarelo = agregado (ag), losango vermelho = ponto de atenção (pa).



A. Notas em aberto do grupo

Captura do board no Miro. Legenda do board: laranja = evento (ev), azul = comando (cmd), lilás = política (pol), verde = modelo de leitura (ml), turquesa = ator (at), rosa = sistema externo (se), amarelo = agregado (ag), losango vermelho = ponto de atenção (pa).



A. Transcrição 1/4: recepção, cadastro e criação da OS

Texto das notas do board, na ordem da linha do tempo, para leitura sem zoom. A primeira coluna traz a nota de ator (at), sistema externo (se) ou modelo de leitura (ml) ligada ao comando; "—" quando o board não indica. Transcrição fiel ao Miro (inclusive termos que o código tratou de outro jeito; ver `docs/ddd/linguagem-ubiqua.md`, seção "Relação com o código").

Ator / modelo de leitura	Comando	Evento	Política / observação
cliente	leva o carro	carro recebido na oficina	
atendente cadastro de cliente	cadastra dados de cliente	dados do cliente cadastrados	
atendente cadastro de veículo	cadastra dados do veículo	dados do veiculo cadastrados	
atendente criação de ordem de serviço	cria ordem de serviço	ordem de serviço criada	"informações: serviços, peças e insumos necessários" notificar mecanico de ordem de serviço criada pa: como é feita essa notificação?

Notas em aberto do grupo (vermelhas no board): "ajustar pra mandar o link pro cliente no momento que os for criada e que os for atualizada com orçamento"; "criar fluxo alternativo quando cliente ja sabe o que precisa ser feito, sem diagnostico".

A. Transcrição 2/4: diagnóstico e orçamento

Ator / modelo de leitura	Comando	Evento	Política / observação
atendente	envia ordem de serviço para mecanico e atualiza status	notificação para análise do veículo e status atualizado	
mecânico	analisa veículo cria relatório	veículo analisado relatório criado	"relatório do serviço" notificar atendente de relatório criado
—	notifica assistente	atendente notificado quanto a criacao de relatorio	
atendente lista de ordem de serviços	atualiza ordem de serviço	ordem de serviço atualizada	"aqui ele busca por cpf/cnpj ou placa do veiculo" gerar orçamento assim que ordem de serviço for atualizada
—	gera orçamento	orçamento inicial gerado	analisa estoque disponivel antes de liberar
estoque	analisa o estoque	estoque disponivel	caso estoque seja insuficiente, adicionar observação a ordem de serviço montar relatorio com peças que tem e o que não tem
atendente	identifica peças faltantes adiciona observação na ordem de serviço	peças faltantes identificadas observação adiciona a ordem de serviço	adicionar observação ao orçamento
atendente ordem de serviço em questão	atualiza ordem de serviço com orçamento baixa o pdf da ordem de serviço	ordem de serviço atualizada pdf baixado	
atendente	envia orçamento para cliente e atualiza status da ordem de serviço	orçamento enviado para cliente por whatsapp e status da ordem de serviço atualizado	"aqui o cliente sabe qual o valor do orçamento, quais peças e serviços devem ser feitos e quais peças temos em estoque..." após 2 dias sem retorno, notificar cliente via whatsapp novamente

A. Transcrição 3/4: aprovação, execução e entrega

Ator / modelo de leitura	Comando	Evento	Política / observação
cliente	aprova orçamento	orçamento aprovado com todas as peças disponíveis	
cliente	aprova parcialmente seja por querer retirar peça indisponivel ou por querer retirar algum serviço/peça disponivel	aprovação parcial	
cliente	nega orçamento	orçamento negado	
—	atualizar estoque	estoque atualizado	uma vez que as peças/insumos são retiradas do estoque, o atendente deve notificar o mecânico
atendente	notifica mecanico e atualiza status da ordem de serviço	mecânico notificado e status ordem de serviço atualizada no sistema	
mecânico	executa serviços descritos na ordem de serviço notifica atendente que veículo está pronto	serviços executados atendente notificado que veículo está pronto	
atendente	notifica cliente envia notificação	cliente é notificado via whatsapp que veiculo está pronto para retirada notificação enviada	

A. Transcrição 4/4: gestão de estoque e agregados

Ator / sistema	Comando	Evento	Política / observação
sistema de ordem de serviço	analisar estoque	estoque analisado	se estoque indisponivel total ou parcialmente, emitir ordem de compra
sistema de compras	gerar ordem de compra		"caso estoque continue insuficiente, ele emite ordem de compra e só segue o pedido após receber"
fornecedor	envia peças / insumos	peças/insumos chegam a mecanica	
atendente cadastro de peças e insumos	cadastra peças e insumos	peças são cadastradas	
—	retorna informação de saldo	informação retornada	

Agregados do board (blocos amarelos, com os comandos e eventos agrupados):

Agregado	Conteúdo no board
Cadastro	cadastro de cliente e de veículo (modelos de leitura e comandos de cadastro)
Ordem de Serviço	criação da OS, envio ao mecânico, relatório, notificações, atualização de status
Relatório	análise do veículo pelo mecânico, criação do relatório e aviso ao atendente
Orçamento	geração do orçamento, análise de estoque, peças faltantes, observação, envio ao cliente
Estoque	análise de saldo, ordem de compra, cadastro de peças e insumos, retorno de saldo

B. Event Storming do sistema implementado: legenda e visão geral

Linhas do tempo pedidas no enunciado (criação da OS, acompanhamento da OS e gestão de peças e insumos), reconstruídas a partir do código. Os eventos são as classes de `ordemservico.domain.evento`; os comandos são os endpoints e casos de uso reais.

ATOR quem dispara | **COMANDO** endpoint / caso de uso | **AGREGADO** quem decide | **EVENTO DE DOMÍNIO** classe publicada | **FATO SEM EVENTO** muda o estado, sem classe de evento | **POLÍTICA** regra automática | **MODELO DE LEITURA** consulta | **SISTEMA EXTERNO** e-mail (simulado)

Evento de domínio	Publicado por	Política que reage
`OrdemServicoCriada`	`CriarOrdemServicoUseCase`	Depois do commit, e-mail simulado ao cliente com o link de consulta e o código de acesso.
`OrcamentoGerado`	`GerarOrcamentoUseCase`	Depois do commit, e-mail simulado avisando que o orçamento está disponível.
`OrdemServicoAprovada`	`AprovarOrcamentoUseCase`	Depois do commit, e-mail simulado de aprovação (total ou parcial).
`OrdemServicoRecusada`	`AprovarOrcamentoUseCase` (sem itens ou expirado), `RejeitarOrcamentoUseCase`	Depois do commit, e-mail simulado "orçamento não aprovado".
`OrdemServicoCancelada`	`CancelarOrdemServicoUseCase`	Depois do commit, e-mail simulado de cancelamento.
`OrdemServicoFinalizada`	`FinalizarOrdemServicoUseCase`	Depois do commit, e-mail simulado "veículo pronto para retirada".

Os eventos são publicados por `PublicadorEventos` e tratados com `@TransactionalEventListener(AFTER_COMMIT)` em `NotificadorOrdemServicoAdapter` e `NotificadorEmailAdapter`. Uma falha no aviso não desfaz a operação. Início da execução, entrega e as mudanças de estoque não publicam evento: ficam no histórico de status (`TransicaoStatus`) ou no saldo do `ProdutoEstoque`, na mesma transação.

B. Linha do tempo 1: criação da OS e orçamento

Do cadastro até o orçamento enviado ao cliente.

1. CADASTROS →

ATOR
Atendente / Admin

COMANDOS
`POST /api/clientes`
`POST /api/veiculos`
`POST /api/servicos`

AGREGADOS
`Cliente`, `Veiculo`, `Servico`

FATOS
cliente, veículo e serviço do catálogo cadastrados

POLÍTICAS
CPF/CNPJ com dígitos válidos e único; placa antiga ou Mercosul e única; nome de serviço único

2. ABRIR A OS →

ATOR
Atendente / Admin

LEITURA
cliente pelo id ou CPF/CNPJ; veículo pelo id ou placa

COMANDO
`POST /api/ordens-servico`
`CriarOrdemServicoUseCase`

AGREGADO
`OrdemServico`

EVENTO
`OrdemServicoCriada`
status `RECEBIDA`

POLÍTICA
o veículo precisa ser do cliente (senão 422)

3. AVISO AO CLIENTE →

POLÍTICA
evento após o commit → e-mail simulado

SISTEMA EXTERNO
e-mail com o link de consulta e o código de acesso (registrado em log)

4. GERAR O ORÇAMENTO →

ATOR
Atendente / Admin (registra o diagnóstico)

LEITURA
`ServicoRepository` (catálogo) e `ProdutoEstoqueRepository`

COMANDO
`POST /api/orcamentos`
`GerarOrcamentoUseCase`

AGREGADOS
`Orcamento`, `OrdemServico`, `ProdutoEstoque`

EVENTO
`OrcamentoGerado`

POLÍTICAS DA GERAÇÃO

POLÍTICA
geração do orçamento → OS `RECEBIDA` passa por `EM_DIAGNOSTICO` e vai para `AGUARDANDO_APROVACAO`

POLÍTICA
item de serviço copia nome e valor do catálogo; alterar o catálogo não muda orçamento antigo

POLÍTICA
peças reservadas no estoque; sem saldo, o orçamento não é gerado

POLÍTICA
evento após o commit → e-mail simulado ao cliente

B. Linha do tempo 2: acompanhamento da OS

Decisão do orçamento, execução, entrega e as saídas de exceção.

1. CONSULTAR →

ATOR
Cliente (sem login)

COMANDO
`POST /api/publico/ordens-servico/{id}/consultar` com CPF/CNPJ e código de acesso

LEITURA
status da OS, veículo e resumo do último orçamento

LEITURA (ATENDENTE)
`GET /api/ordens-servico` e `/{id}`

2. APROVAR →

ATOR
Cliente (`/api/publico/orcamentos/{id}/aprovar`) ou Atendente (`PATCH /api/orcamentos/{id}/aprovar`)

COMANDO
`AprovarOrcamentoUseCase`, total ou com itens aprovados

AGREGADOS
`Orcamento`, `OrdemServico`, `ProdutoEstoque`

EVENTO
`OrdemServicoAprovada`
OS `APROVADA`

POLÍTICA
aprovação → baixa das reservas dos itens aprovados; itens recusados devolvem a reserva

POLÍTICA
orçamento expirado ao aprovar → orçamento `EXPIRADO`, OS recusada e reservas devolvidas (`OrdemServicoRecusada`)

3. EXECUTAR →

ATOR
Atendente / Admin

COMANDO
`PATCH /api/ordens-servico/{id}/iniciar-execucao`

FATO
OS `EM_EXECUCAO` (histórico)

COMANDO
`PATCH .../finalizar`

EVENTO
`OrdemServicoFinalizada`

SISTEMA EXTERNO
e-mail "veículo pronto"

4. ENTREGAR →

ATOR
Atendente / Admin

COMANDO
`PATCH .../entregar`

FATO
OS `ENTREGUE` (histórico)

LEITURA
histórico de status → tempo médio de execução (consulta em desenvolvimento no #42)

EXCEÇÕES

COMANDO
rejeitar: `/api/publico/orcamentos/{id}/rejeitar` ou `PATCH /api/orcamentos/{id}/rejeitar`

EVENTO
`OrdemServicoRecusada`; orçamento `REJEITADO`

POLÍTICA
aprovação sem nenhum item → `REJEITADO` e OS recusada

COMANDO
`PATCH /api/ordens-servico/{id}/cancelar` (antes da aprovação)

EVENTO
`OrdemServicoCancelada`

POLÍTICA
rejeição/cancelamento → devolução das reservas

Aprovação, recusa, cancelamento e finalização avisam o cliente por e-mail simulado depois do commit. Início da execução e entrega ficam só no histórico de status.

B. Linha do tempo 3: gestão de peças e insumos

Cadastro e saldo do estoque, e como o fluxo da OS consome o estoque. O estoque não publica eventos de domínio: as mudanças são feitas na mesma transação de quem chama.

1. CADASTRAR →

ATOR
Atendente / Admin

COMANDO
cadastrar peça ou insumo

AGREGADO
`ProdutoEstoque`

FATO
peça cadastrada com saldo disponível inicial

POLÍTICA
nome da peça único; valor unitário maior que zero

2. AJUSTAR SALDO →

ATOR
Atendente / Admin

COMANDOS
entrada e saída manual de quantidade

FATO
saldo disponível alterado

POLÍTICA
a saída manual não mexe no reservado; reservas são só do orçamento

3. CONSULTAR / MANTER →

LEITURA
consulta de peça com saldo disponível e reservado

COMANDOS
listagem, atualização e exclusão de peças (em desenvolvimento no #43)

4. CONSUMO PELA OS

GATILHO
geração do orçamento (`OrcamentoGerado`)

FATO
`reservar`: disponível → reservado

GATILHO
`OrdemServicoAprovada`

FATO
`confirmarBaixa`: sai do reservado

GATILHO
`OrdemServicoRecusada` / `OrdemServicoCancelada` / item recusado

FATO
`cancelarReserva`: reservado → disponível

Os endpoints de estoque estão no #43 e podem mudar até o merge, por isso esta página usa nomes de ação. Os métodos `reservar`, `confirmarBaixa` e `cancelarReserva` já estão na `dev`.

B. Agregados e onde cada evento nasce

Agregado (raiz)	Comandos que recebe	Eventos / fatos	Invariantes
`OrdemServico`	criar, iniciar diagnóstico (automático), anexar orçamento, aprovar, recusar, cancelar, iniciar execução, finalizar, entregar	`OrdemServicoCriada`, `OrdemServicoAprovada`, `OrdemServicoRecusada`, `OrdemServicoCancelada`, `OrdemServicoFinalizada`; `TransicaoStatus` a cada mudança	só transições permitidas; cancelamento só antes da aprovação; relato de 10 a 500 caracteres
`Orcamento`	gerar, aprovar (total/parcial), rejeitar	`OrcamentoGerado`; status `APROVADO`, `APROVADO_PARCIALMENTE`, `REJEITADO`, `EXPIRADO`	só muda enquanto `PENDENTE`; validade no futuro; aprovar depois da validade expira
`ProdutoEstoque`	cadastrar, entrada, saída, reservar, confirmar baixa, cancelar reserva	fatos de saldo (sem evento)	reservar só com saldo; baixa e devolução só sobre o reservado
`Cliente`	CRUD	fatos de cadastro (sem evento)	CPF/CNPJ válido e único; não excluir com veículos
`Veiculo`	CRUD	fatos de cadastro (sem evento)	placa válida e única; não excluir com OS
`Servico`	CRUD	fatos de cadastro (sem evento)	nome único; valor maior que zero; não excluir se já usado em orçamento

Comparação com o board exploratório: o Miro tinha os agregados Cadastro, Relatório, Orçamento, Estoque e Ordem de Serviço. No código, Cadastro virou `Cliente` e `Veiculo`, Relatório foi absorvido pelo orçamento e o catálogo virou `Servico`. As diferenças estão em `docs/ddd/linguagem-ubiqua.md`, seção "Relação com o código".